

Robot Programming Experience

Using developers' mental models to improve their robot programming experience

Miguel Campusano

mica@mmmi.sdu.dk

SDU Software Engineering



Programming Experience

*Do programmers create text that is transformed into **running behavior** (the old way), or do they operate on **behavior directly** ("liveness"); are they exploring the **live domain** to understand the true nature of the requirements; are they like authors creating **new worlds**; **does visualization matter**; is the experience **immediate, immersive, vivid and continuous**; do **fluency, literacy, and learning** matter; do they **build tools, meta-tools**; are they creating **languages** to express **new concepts quickly and easily**; and curiously, is **joy** relevant to the experience?*

- Programming Experience Workshop

Robot Programming Experience

Programming Experience

User eXperience (UX) applied for programming (PX)

UX: users interaction with systems

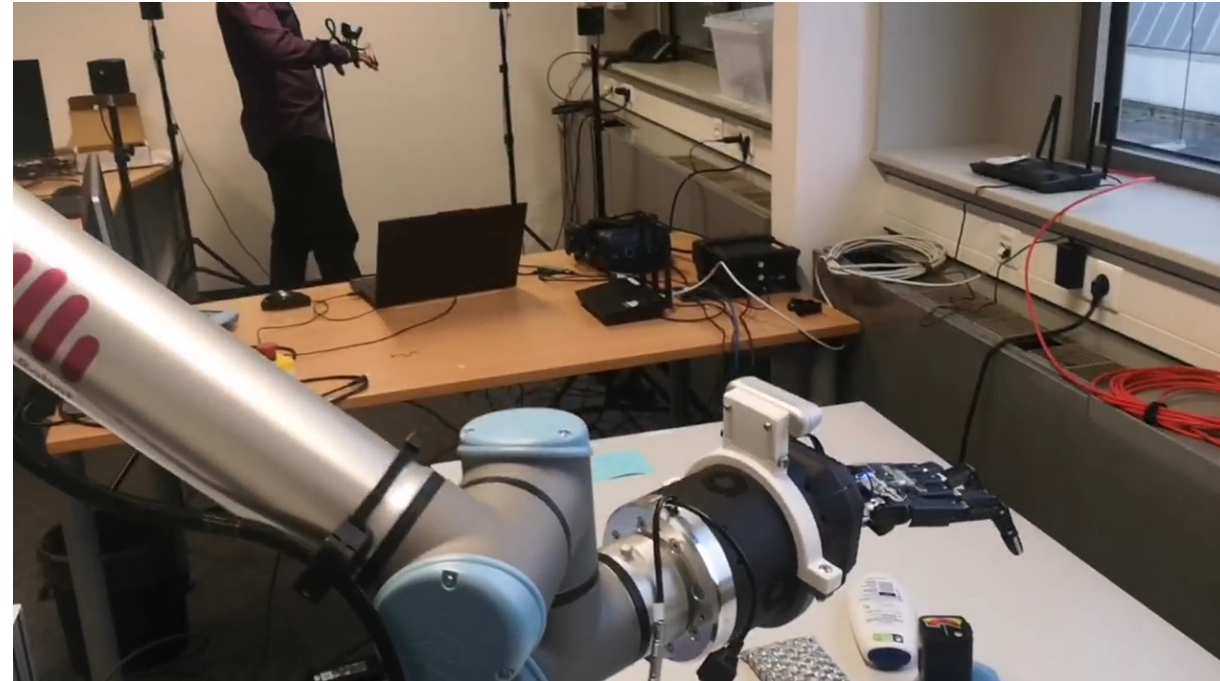
PX: *programmer* interaction with programming systems

Robot Programming Experience (RPX)

PX for robotic development

Programmer

Developer, Operators, Users, etc.



Programming: Difficult => Inefficient

Programming Robots is difficult

Combination actuators and sensors

Deep expertise in diverse areas

Robotic developers are not trained as developers

...

Then, development of robot programs

Takes more time

Requires extra expertise

Is blurry => not sure when robot will work



RPX with Mental Models

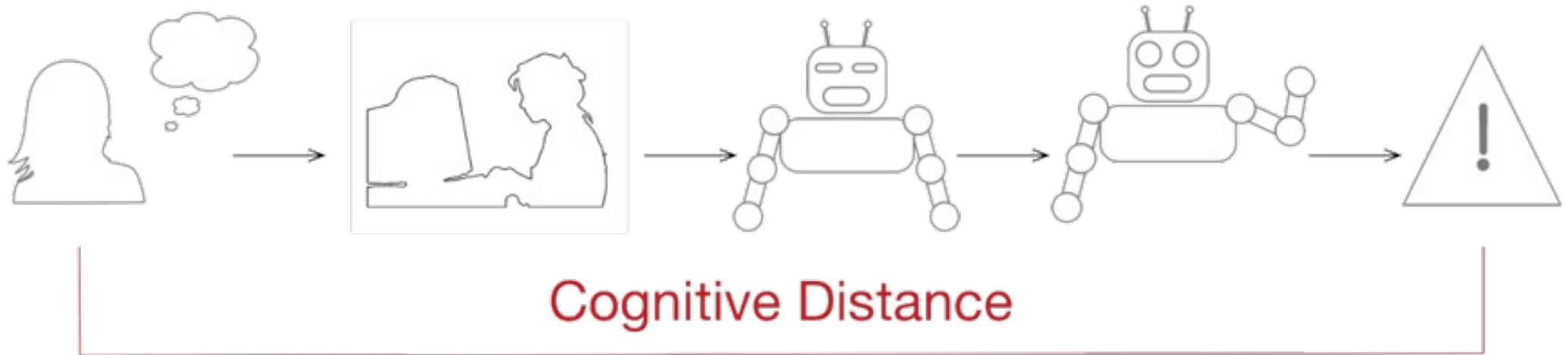
Mental Model: Developer's thoughts

Thoughts into code, then action, then *errors*

Use mental models to improve RPX

Mental Model $\Leftrightarrow$ Computational Model

Coding, Understanding, Debugging, ...



State Machines

Explicit mental/computation model

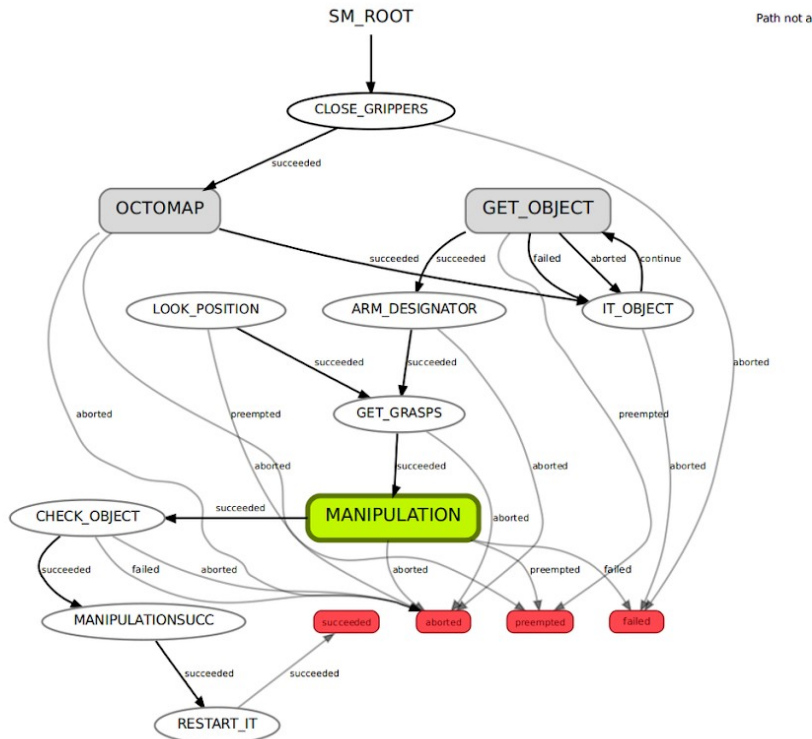
Understanding State Machines

Static representation / Program

Dynamic representation?

Understand execution

Solve bugs



```
/home/miguel/catkin_ws/src/ros_exercise/launch/husky.launch http://127.0.0.1:11311/
Released under the Apache 2 License.
http://gazebo.in.org

Msg Waiting for master. [ INFO ] [1548960025.535857875]: Stereo is NOT SUPPORTED
[ INFO ] [1548960025.535923330]: OpenGL version: 3 (GLSL 1.3).
[ INFO ] [1548960025.557097549]: Finished loading Gazebo ROS API Plugin.
Msg Waiting for master
[ INFO ] [1548960025.558525830]: waitForService: Service [/gazebo/set_physics_properties] has not been advertised, waiting...
Msg Connected to gazebo master @ http://127.0.0.1:11345
Msg Publicized address: 172.17.69.79
Warning [parser_urdf.cc:1010] multiple inconsistent <gravity> exists due to fixed joint reduction overwriting previous value [true] with [false].

Msg Connected to gazebo master @ http://127.0.0.1:11345
Msg Publicized address: 172.17.69.79
[spawn_husky_model-9] process has finished cleanly
log file: /home/miguel/.ros/log/abdf6d42-2587-11e9-9551-4ccc6ab0e9a3/spawn_husky_model-9*.log
Error [Param.cc:181] Unable to set value [1.0471975511965976] for key[horizontal_fov]
Error [Param.cc:181] Unable to set value [0.1000000001] for key[near]
[ INFO ] [1548960026.799408590, 1298.480000000]: Laser Plugin (robotNamespace = /), Info: Using the 'robotNamespace' param: '/'
[ INFO ] [1548960026.799474233, 1298.480000000]: Starting Laser Plugin (ns = /)
[ INFO ] [1548960026.800655387, 1298.480000000]: Laser Plugin (ns = /) <tf_prefix>, set to ""
[ INFO ] [1548960026.818945374, 1298.480000000]: Loading gazebo_ros_control plugin
[ INFO ] [1548960026.819111283, 1298.480000000]: Starting gazebo_ros_control plugin in namespace: /
[ INFO ] [1548960026.81936712, 1298.480000000]: gazebo_ros_control plugin is waiting for model URDF in parameter [robot_description] on the ROS param server.
[ INFO ] [1548960026.963058118, 1298.480000000]: Loaded gazebo_ros_control.
[ INFO ] [1548960026.965205242, 1298.480000000]: imu plugin missing <xyzOffset>, defaults to 0s
[ INFO ] [1548960026.965230776, 1298.480000000]: imu plugin missing <rpzOffset>, defaults to 0s
[ INFO ] [1548960027.025394918, 1298.480000000]: Controller state will be published at 50Hz.
[ WARN ] [1548960027.026208458, 1298.490000000]: Tuned out waiting for transform from base_link to odom to become available before running costmap, tf error: canTransform: target_frame odom does not exist..
canTransform returned after 1298.49 timeout was 0.1.
[ INFO ] [1548960027.027021129, 1298.490000000]: Wheel separation will be multiplied by 1.875.
[ INFO ] [1548960027.027594402, 1298.490000000]: Wheel radius will be multiplied by 1.
[ INFO ] [1548960027.028303855, 1298.490000000]: Velocity rolling window size of 10.
[ INFO ] [1548960027.029172953, 1298.490000000]: Velocity commands will be considered old if they are older than 0.25s.
[ INFO ] [1548960027.029698551, 1298.490000000]: Allow multiple cmd_vel publishers is enabled
[ INFO ] [1548960027.032585394, 1298.490000000]: Base frame_id set to base_link
[ INFO ] [1548960027.032879566, 1298.490000000]: Odometry frame_id set to odom
[ INFO ] [1548960027.033487138, 1298.490000000]: Publishing to tf is disabled
[ INFO ] [1548960027.037455065, 1298.500000000]: waitForService: Service [/gazebo/set_physics_properties] is now available.
[ INFO ] [1548960027.050846249, 1298.510000000]: left wheel to origin: 0.256,-0.2854, 0.03282
[ INFO ] [1548960027.050877957, 1298.510000000]: right wheel to origin: 0.256,-0.2854, 0.03282
[ INFO ] [1548960027.050964697, 1298.510000000]: Odometry params : wheel separation 1.07025, wheel radius 0.1651
[ INFO ] [1548960027.054621904, 1298.510000000]: Adding left wheel with joint name: front_left_wheel and right wheel with joint name: front_right_wheel
[ INFO ] [1548960027.054700958, 1298.510000000]: Adding left wheel with joint name: rear_left_wheel and right wheel with joint name: rear_right_wheel
[ INFO ] [1548960027.066802465, 1298.520000000]: Physics dynamic reconfigure ready.
[ INFO ] [1548960027.190585253, 1298.650000000]: Using plugin "obstacles_laser"
[ INFO ] [1548960027.220496278, 1298.680000000]: Subscribed to Topics: laser
[ INFO ] [1548960027.240577908, 1298.700000000]: Using plugin "inflation"
[ INFO ] [1548960027.310867177, 1298.770000000]: Using plugin "obstacles_laser"
[ INFO ] [1548960027.340280319, 1298.800000000]: Subscribed to Topics: laser
[ INFO ] [1548960027.355739550, 1298.810000000]: Using plugin "inflation"
[ INFO ] [1548960027.410049458, 1298.870000000]: Created local_planner dwa_local_planner/DWAPlannerROS
[ INFO ] [1548960027.413289992, 1298.870000000]: Sm period is set to 0.20
Laser Poses: 0.337005 3.68011e-05 0.000109438
[ INFO ] [1548960027.658823279, 1299.110000000]: Recovery behavior will clear layer obstacles
[ INFO ] [1548960027.691797140, 1299.150000000]: Recovery behavior will clear layer obstacles
```

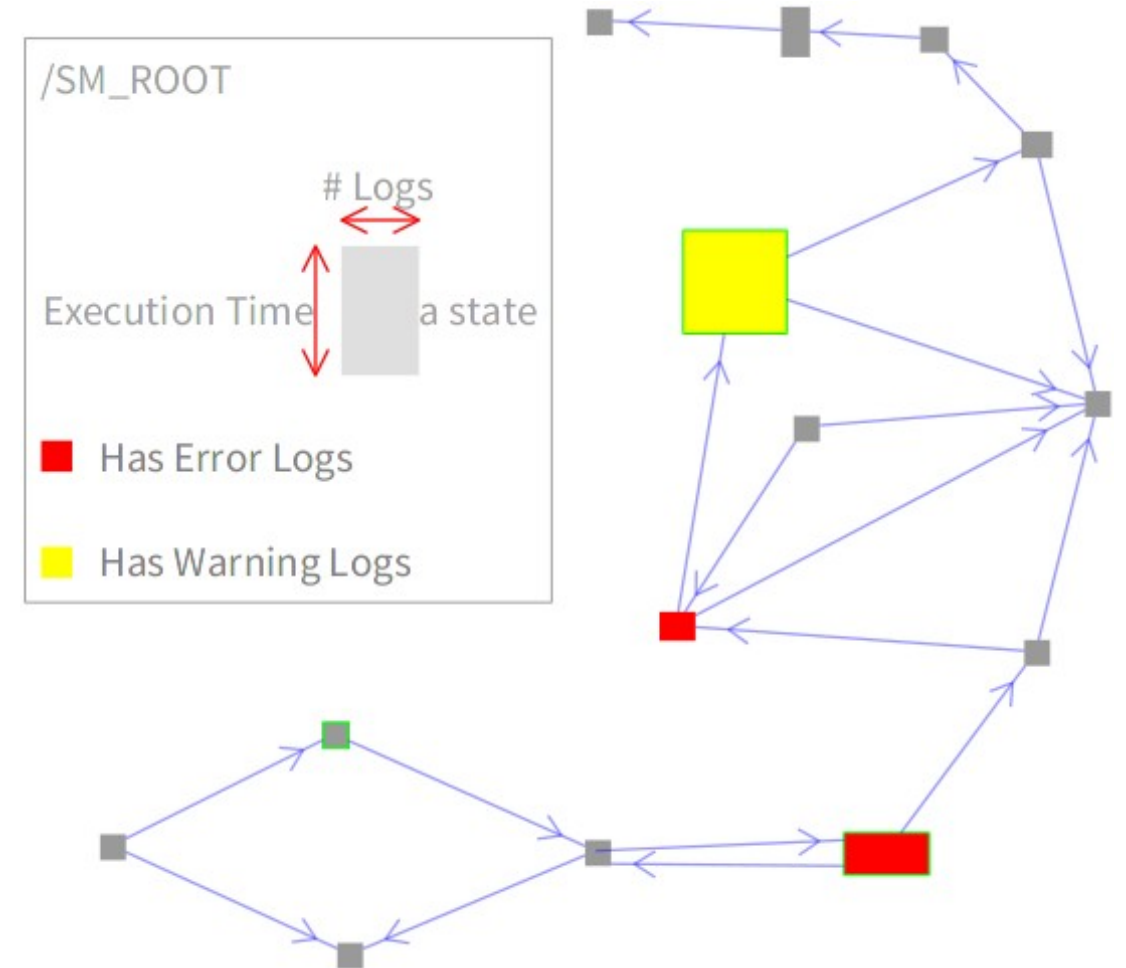
Understanding State Machines

Static representation / Program

Dynamic representation?

Understand execution

Solve bugs



Understanding State Machines

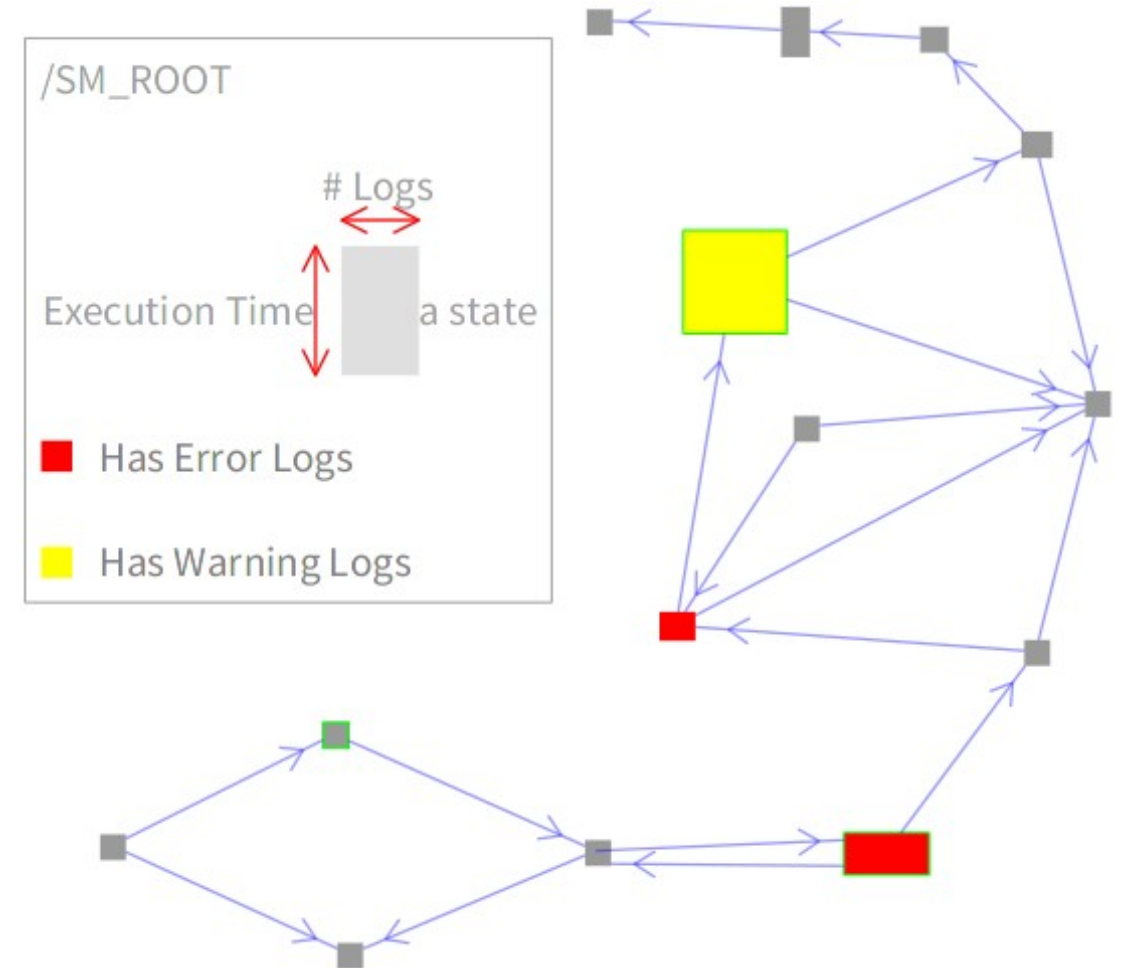
Static representation / Program

Dynamic representation?

Understand execution

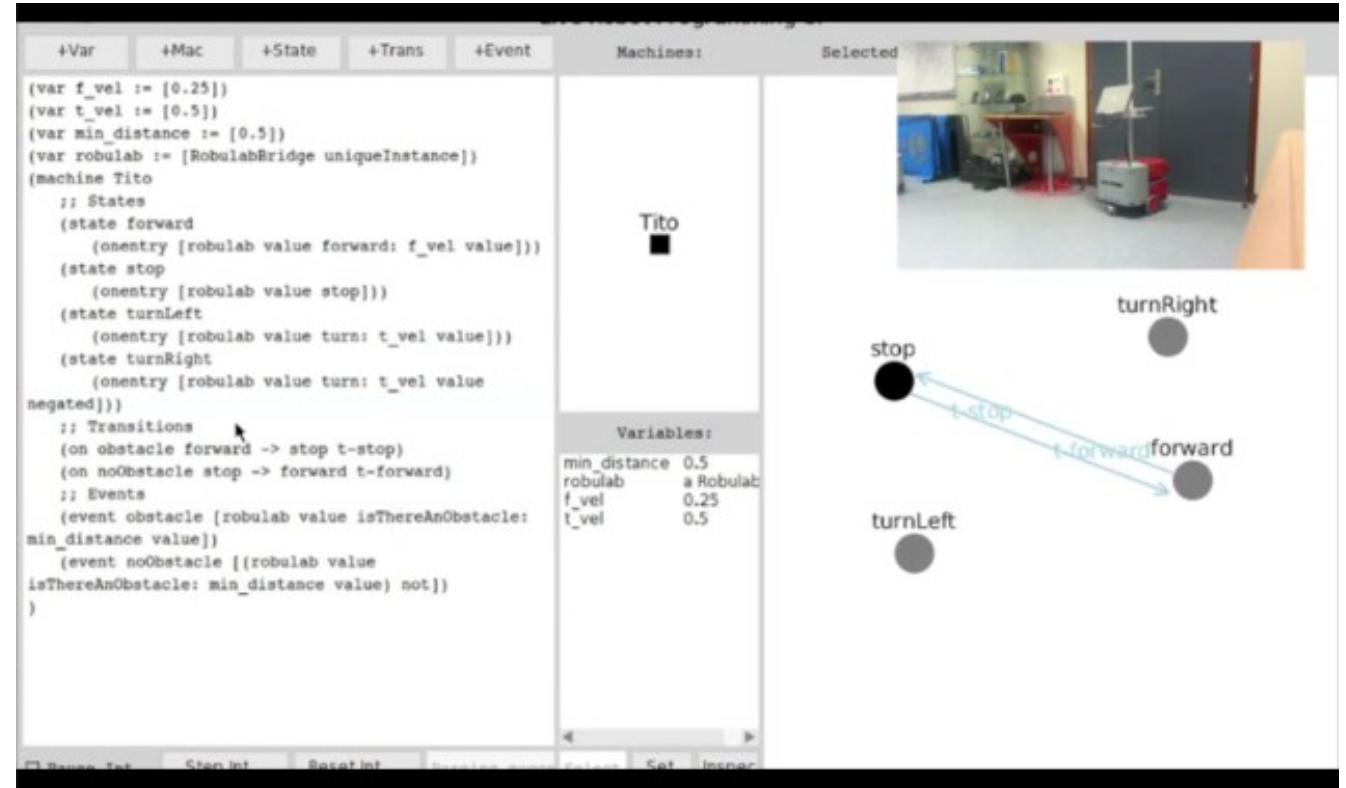
Solve bugs

It helps finding *hard-to-find* bugs



Programming State Machines

Using domain-specific properties



Towards Live Programming in ROS with PhaROS and LRP – 2014

https://youtu.be/svgYA1_Dpjl?si=Cb-qNgjYil0tbSJH

Programming State Machines

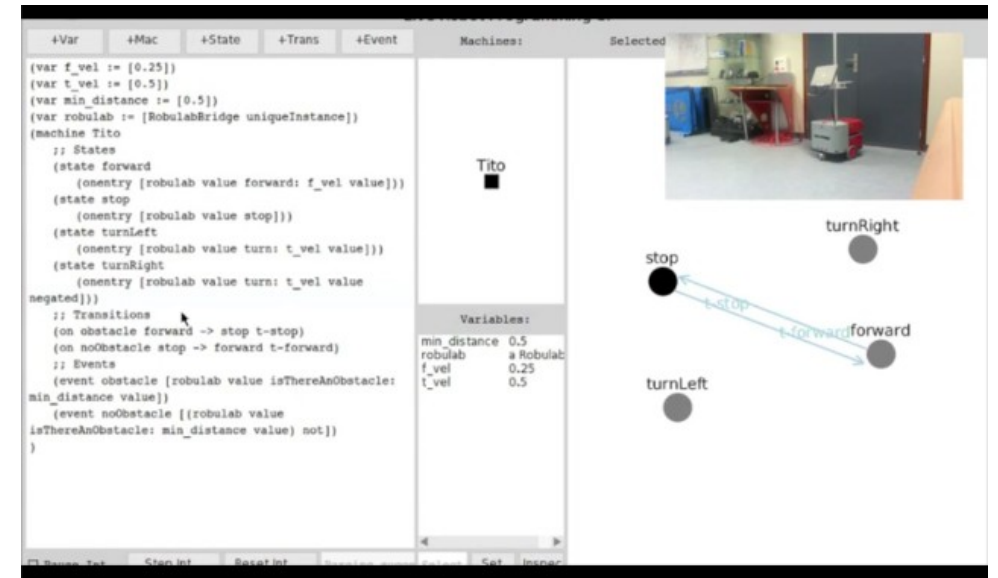
Using domain-specific properties

It does NOT outperform baseline (correct and time)

But devs prefer it

Is there a need to build simpler APIs?

Maybe current layer is too complex...



Live programming in practice: A controlled experiment on state machines for robotic behaviors - 2019

Imperative Language

Implicit mental/computation model

Understanding in Imperative Language

Static representation

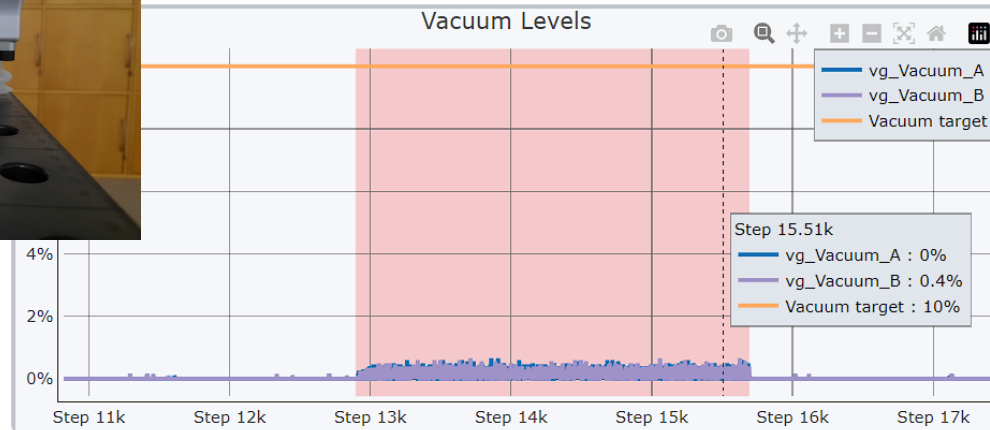
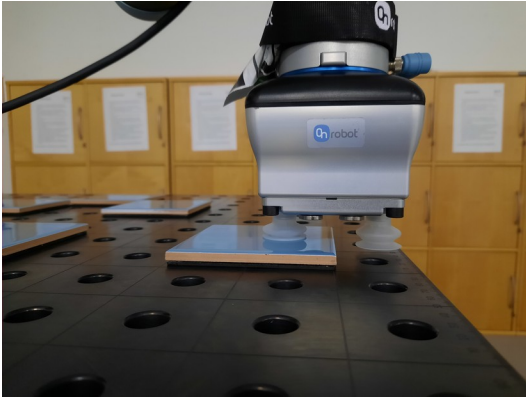
```
script
136  vg10_grip(2, vacuum_level, 0, False)
137
138  # Lift up to stand clear
139  pickup_pose[2] = travel_height
140  move1(pickup_pose, a=move_acceleration, v=move_speed)
141
142  # Move to the final/dropoff position (Should be movej, but because of qnear shenanigan
143  move1(dropoff_pose, r=0.1)
144
145  dropoff_lowered = dropoff_pose
146  dropoff_lowered[2] = dropoff[2]
147  # Move straight down to drop off the item
148  move1(dropoff_lowered)
149
150  # Open the tool
151  #popup("Drop the vacuum here!", "Dropping off the tile", False, False, True)
152
153  # channel, timeout, autoidle, toolindex
154  # Channel: 0 = A, 1 = B, 2 = Both
155  # timeout is how long the command waits for the vacuum to drop, before giving an error
156  # it is given in seconds : 0.6 = 600 ms
157  # autoidle :
158  # Whether the release valve should be automatically turned off when the release is
159  # and the robot has moved 5 cm away from the release position
160  # tool index is only applicable if more than one gripper is mounted
161  vg10_release(2, 0.6)
```

Understanding in Imperative Language

Static representation with runtime data

Understand execution

Solve bugs



```
script

136  vg10_grip(2, vacuum_level, 0, False)
137
138  # Lift up to stand clear
139  pickup_pose[2] = travel_height
140  movej(pickup_pose, a=move_acceleration, v=move_speed)
141
142  # Move to the final/dropoff position (Should be movej, but because of qnear shenanigan
143  movej(dropoff_pose, r=0.1)
144
145  dropoff_lowered = dropoff_pose
146  dropoff_lowered[2] = dropoff[2]
147  # Move straight down to drop off the item
148  movej(dropoff_lowered)
149
150  # Open the tool
151  #popup("Drop the vacuum here!", "Dropping off the tile", False, False, True)
152
153  # channel, timeout, autoidle, toolindex
154  # Channel: 0 = A, 1 = B, 2 = Both
155  # timeout is how long the command waits for the vacuum to drop, before giving an error
156  # it is given in seconds : 0.6 = 600 ms
157  # autoidle :
158  # Whether the release valve should be automatically turned off when the release is
159  # and the robot has moved 5 cm away from the release position
160  # tool index is only applicable if more than one gripper is mounted
161  vg10_release(2, 0.6)
```

Contextual visualizations for debugging collaborative robots - 2024

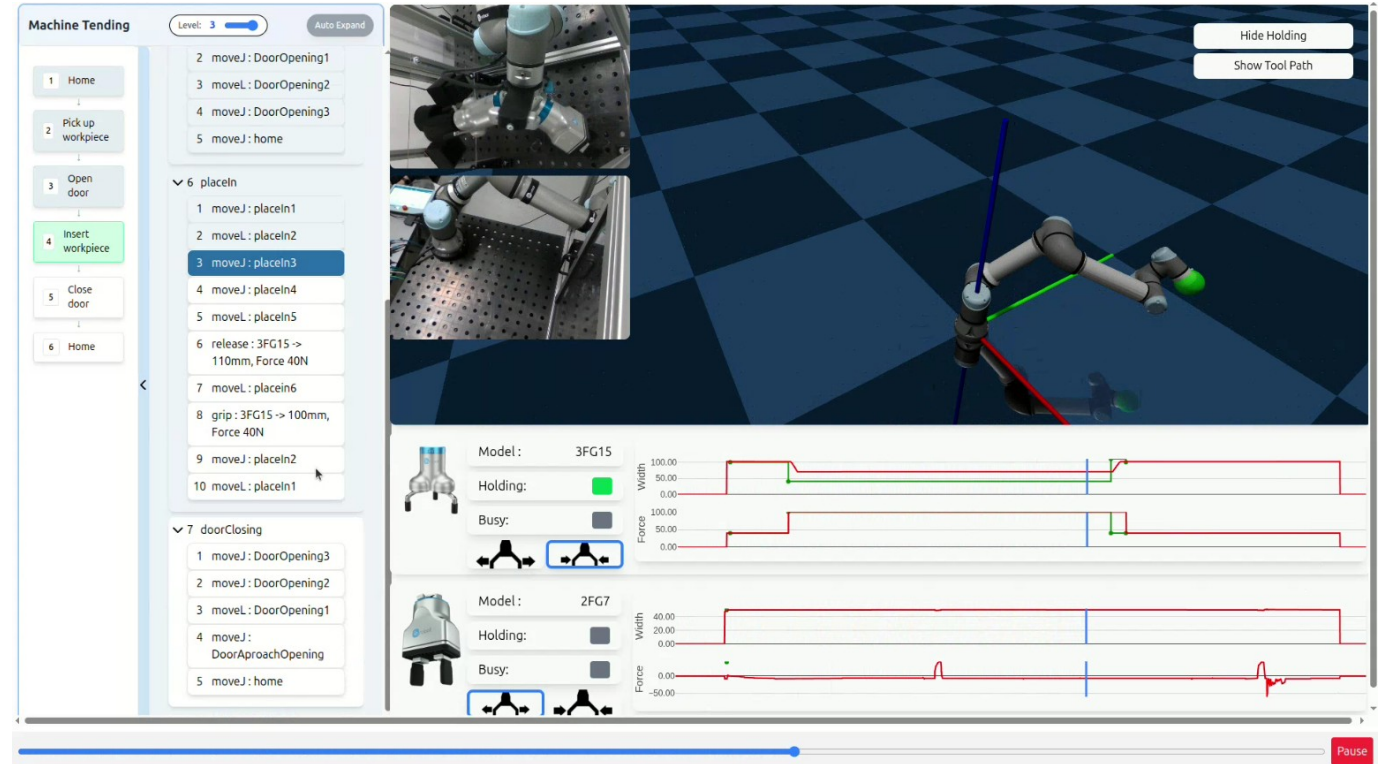
<https://youtu.be/FffedGoe-fU>

Understanding in Imperative Language

Static representation with runtime data

Include domain-specific model

Robot Arm / Machine Tending



Complex models

Physical and Software

Soft Robots

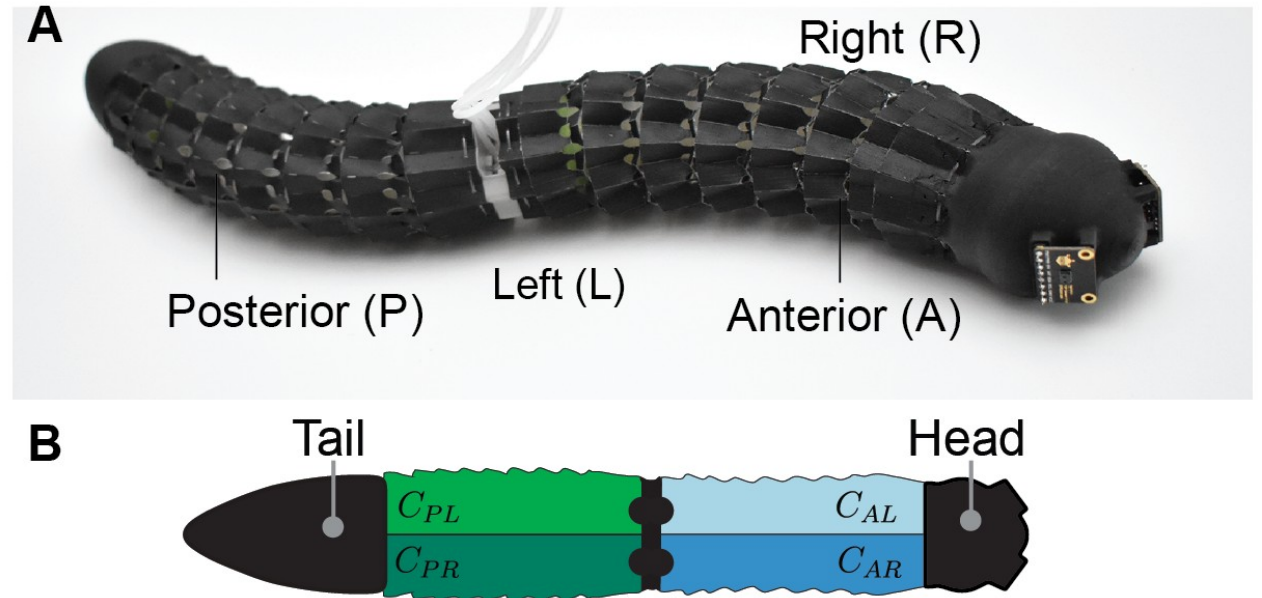
Compliant Materials

Instead of rigid bodies

Infinite degrees of freedom

Complex to reason within code

Complex to test



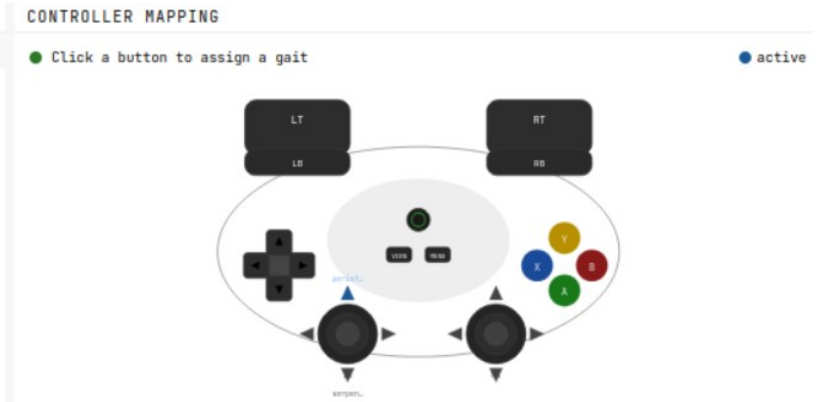
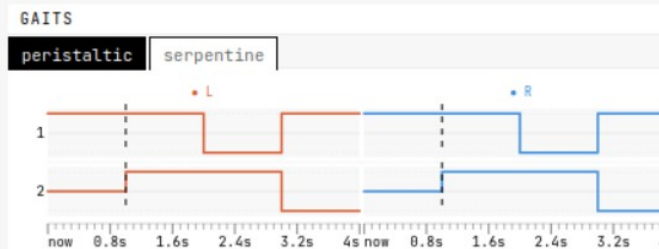
<https://youtu.be/yLssUETKmHA?si=upcO7jqCQR4qeGVS>

Soft Robots: Models and Runtime Feedback

DSL

LSP connected

```
1 2 segments
2 chambers: left, right, both
3 tick: 500 ms
4
5 mapping:
6   inflate:
7     pump: 255
8     left: 1, 0, _
9     right: _, 0, 1
10    both: 1, 0, 1
11   deflate:
12     pump: 0
13     left: 0, 1, _
14     right: _, 1, 0
15     both: 0, 1, 0
16   hold:
17     pump: 0
18     left: 1, _, _
19     right: _, _, 1
20     both: 0, 0, 0
21
22 signal sigPer1:
23   inflate both for 4 ticks
24   deflate both for 2 ticks
25 signal sigSer1:
26   inflate left for 4 ticks
27   deflate left for 2 ticks
28   inflate right for 4 ticks
29   deflate right for 2 ticks
30
31 gait peristaltic:
32   1: sigPer1
33   2: sigPer1 offset(2)
34 gait serpentine:
35   1: sigSer1
36   2: sigSer1 offset(2)
37
38 buttons:
39   LS_UP -> peristaltic
40   LS_DOWN -> serpentine
41
```



RPX

What is next

RPX: Does it work?

Depends

Some studies do not show increase in productivity

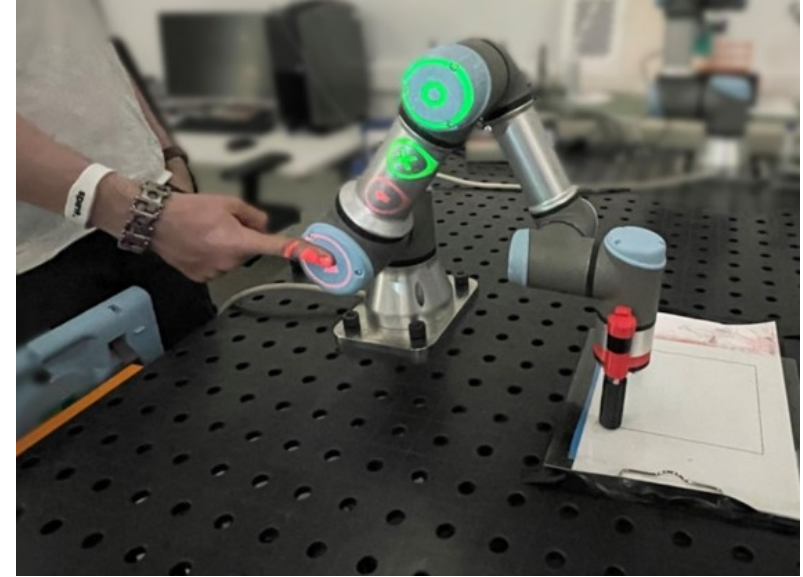
Others show they can help finding difficult bugs

But they usually show that developers *prefer* them

Models are always useful

Improving experience can be helpful

Improving experience can reduce *cognitive load* (or increase joy?)



RPX: current work

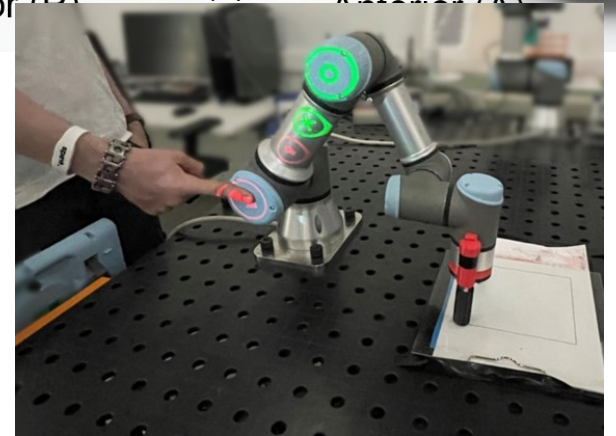
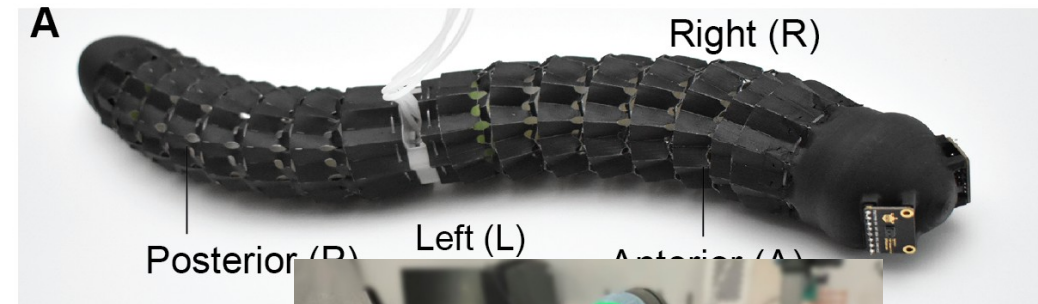
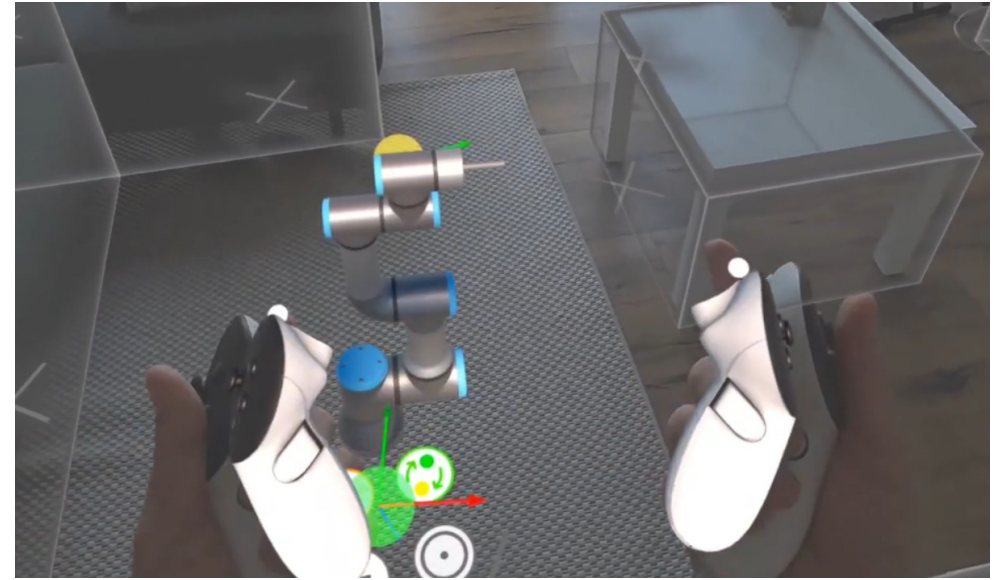
Programming robots with XR

Live programming and soft robots

Live and reconfigurable languages

Re-program collaborative robots

We see them, now we program them



RPX: current work

Programming robots with XR

Live programming and soft robots

Live and reconfigurable languages

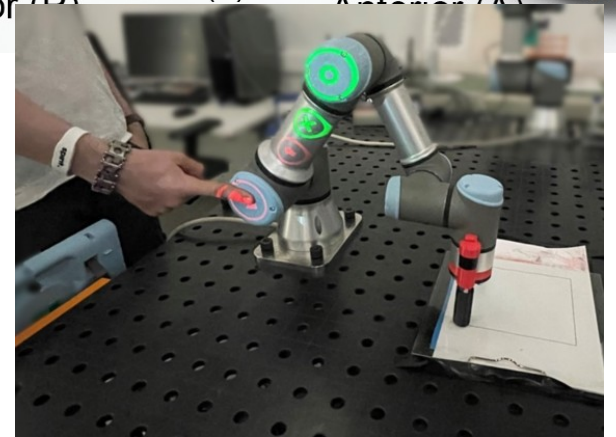
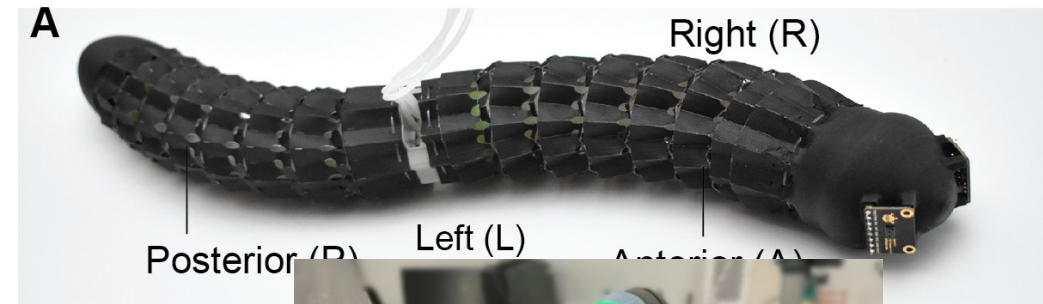
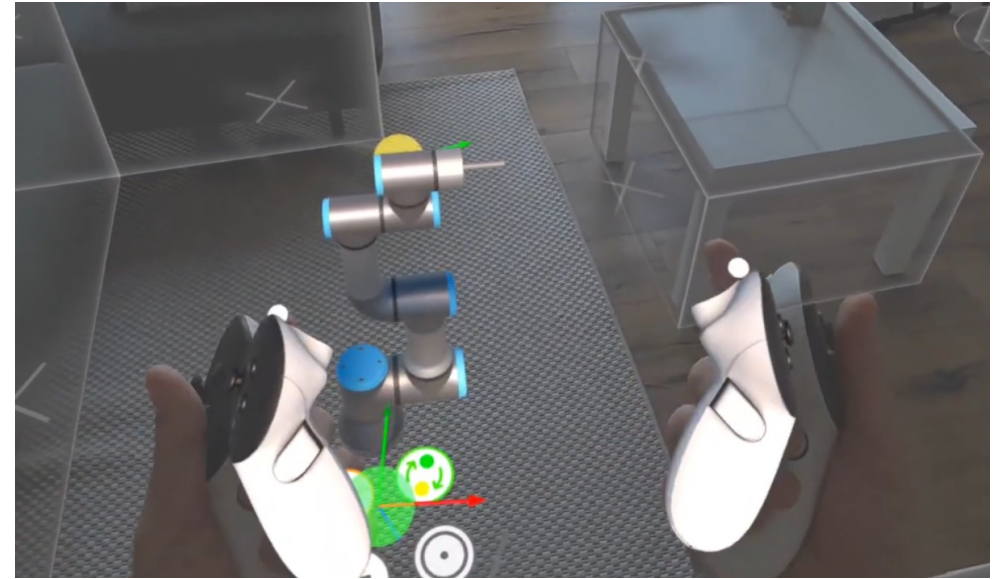
Re-program collaborative robots

We see them, now we program them

Do you *need* RPX?

Computational Models (behavior trees)

Robots (underwater, vehicles, HRI)



Robot Programming Experience

Using developers' mental models to improve their robot programming experience

Miguel Campusano

mica@mmmi.sdu.dk

SDU Software Engineering

